LINKED LIST

Implementation in Python

TOPICS COVERED

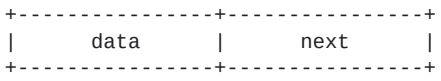
- Introduction and structure of a singly linked list
- Node creation and connecting nodes
- Creating a LinkedList class
- Traversal and display
- Insertion at the beginning and at the end
- Deletion from the beginning and from the end
- Complete Python implementation
- Python's predefined alternative: collections.deque
- Time complexity and practice questions

1. Introduction to Linked List

A **Linked List** is a linear data structure in which elements are stored in separate objects called **nodes**. Each node stores data and a reference to the next node. Nodes are connected using these references.

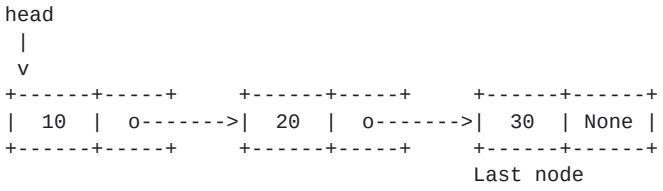
Unlike a Python list (which is implemented as a dynamic array), linked-list nodes do not need to be stored next to one another in memory.

Structure of a node



data: Stores the value. **next:** Stores a reference to the next node.

Example of a singly linked list



• **Head:** Reference to the first node in the list.

• **Next:** Link/reference that connects one node to the next node.

• **None:** Shows that there is no next node. The last node points to None.

> **Remember**
> A linked list is not accessed by directly jumping to an index like a Python list. We normally start at head and move from node to node.

2. Step 1 – Creating a Node

The first step is to define a class named **Node**. The constructor receives a value and stores it in data. The next reference is initially set to None because the node is not connected yet.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


n1 = Node(10)
n2 = Node(20)
n3 = Node(30)

print(n1.data)
print(n1.next)
```

Output

```
10
None
```

Explanation

• The statement Node(10) creates a new node containing 10.

• self.data = data stores the value passed to the constructor.

• self.next = None means that the node is not pointing to another node yet.

• At this stage, n1, n2 and n3 are separate nodes. They are not connected.

3. Step 2 – Connecting Nodes Manually

To connect nodes, assign the next reference of one node to the next node. The variable head stores the reference to the first node.

```
n1.next = n2
n2.next = n3

head = n1
```

```
head
 |
 v
[10]  ->  [20]  ->  [30]  ->  None
```

Now the nodes form a chain. The list can be accessed starting from head.

Try traversing the connected nodes

```
temp = head

while temp != None:
    print(temp.data)
    temp = temp.next
```

Output: 10, 20, 30 (each value appears on a separate line).

The variable temp moves from the current node to the next node until it becomes None. This process is called **traversal**.

4. Step 3 – Creating a LinkedList Class

Creating nodes manually is useful for understanding references, but it is not convenient for managing a list. We create a LinkedList class to manage the head and the operations performed on the list.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None


mylist = LinkedList()
```

When a LinkedList object is created, head is set to None. This means the list is empty.

Important difference
Node represents one element. LinkedList represents and manages the complete chain of nodes.

5. Step 4 – Displaying the Linked List (Traversal)

Traversal means visiting each node one by one, starting from the head. A temporary variable is used so that the head reference is not changed.

```
def display(self):
    temp = self.head

    while temp != None:
        print(temp.data, end=" -> ")
        temp = temp.next
```

```
    print("None")
```

If the list contains 10, 20 and 30, the display will be:

```
10 -> 20 -> 30 -> None
```

• temp = self.head starts from the first node.

• temp.data reads the current node's value.

• temp = temp.next moves to the next node.

• The loop stops when temp becomes None.

6. Step 5 – Insertion at the Beginning

Insertion means adding a new node to the linked list. To insert at the beginning, the new node must point to the current first node. Then head is updated to point to the new node.

```
def insert_beginning(self, data):
    newnode = Node(data)

    newnode.next = self.head
    self.head = newnode
```

Example

```
Before:  head -> 20 -> 30 -> None

Insert 10 at the beginning

After:   head -> 10 -> 20 -> 30 -> None
```

If the list is empty, self.head is None. The new node's next reference becomes None, and head is updated to the new node. Therefore, the same code also works for an empty list.

Key point
Update newnode.next first, and then update self.head. This preserves the link to the existing list.

7. Step 6 – Insertion at the End

To insert at the end, first create a new node. If the list is empty, make it the head. Otherwise, move through the list until the last node is reached. The last node is identified by temp.next == None.

```
def insert_end(self, data):
    newnode = Node(data)

    if self.head == None:
        self.head = newnode
    else:
        temp = self.head

        while temp.next != None:
            temp = temp.next

        temp.next = newnode
```

Example

```
Before:  head -> 10 -> 20 -> None

Insert 30 at the end

After:   head -> 10 -> 20 -> 30 -> None
```

• The if condition handles an empty linked list.

• The while loop moves temp to the last node.

• temp.next = newnode connects the last node to the newly created node.

8. Step 7 – Deletion from the Beginning

Deletion means removing a node from the linked list. To delete the first node, move head to the second node. The old first node is no longer part of the list.

```
def delete_beginning(self):
    if self.head == None:
        print("List is empty")
    else:
        self.head = self.head.next
```

```
Before:  head -> 10 -> 20 -> 30 -> None
After:   head -> 20 -> 30 -> None
```

If the list is empty, there is nothing to delete. If the list contains only one node, self.head.next is None, so head becomes None and the list becomes empty.

9. Step 8 – Deletion from the End

To delete the last node, we need to reach the second-last node and set its next reference to None. Special cases are required for an empty list and a list containing only one node.

```
def delete_end(self):
    if self.head == None:
        print("List is empty")

    elif self.head.next == None:
        self.head = None

    else:
        temp = self.head

        while temp.next.next != None:
            temp = temp.next

        temp.next = None
```

```
Before:  head -> 10 -> 20 -> 30 -> None
After:   head -> 10 -> 20 -> None
```

The loop stops when temp is at the second-last node. Setting temp.next to None disconnects the last node.

10. Complete Program – Singly Linked List

The following program combines the operations covered in these notes. Type it into a Python file and run it.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def insert_beginning(self, data):
        newnode = Node(data)
        newnode.next = self.head
        self.head = newnode

    def insert_end(self, data):
        newnode = Node(data)

        if self.head == None:
            self.head = newnode
        else:
            temp = self.head

            while temp.next != None:
                temp = temp.next

            temp.next = newnode

    def delete_beginning(self):
        if self.head == None:
            print("List is empty")
        else:
            self.head = self.head.next

    def delete_end(self):
        if self.head == None:
            print("List is empty")
        elif self.head.next == None:
            self.head = None
        else:
            temp = self.head

            while temp.next.next != None:
                temp = temp.next

            temp.next = None

    def display(self):
        temp = self.head

        while temp != None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


# Main program
l = LinkedList()

l.insert_beginning(20)
l.insert_beginning(10)
l.insert_end(30)
l.insert_end(40)

print("Original list:")
l.display()
```

```
l.delete_beginning()
print("After deleting first node:")
l.display()

l.delete_end()
print("After deleting last node:")
l.display()
```

Expected output

```
Original list:
10 -> 20 -> 30 -> 40 -> None
After deleting first node:
20 -> 30 -> 40 -> None
After deleting last node:
20 -> 30 -> None
```

11. Is There a Predefined Linked List in Python?

Python does not provide a built-in LinkedList class in its standard library. For learning how a linked list works internally, we create our own Node and LinkedList classes.

Python does provide **collections.deque**. It is a double-ended queue implemented using a linked structure of blocks. It is useful when elements need to be added or removed efficiently from both ends, but it does not expose individual linked-list nodes or next references.

```
from collections import deque

items = deque()

# Add elements
items.append(10)
items.append(20)
items.append(30)

# Add an element at the beginning
items.appendleft(5)

print(items)

# Remove from the beginning and end
items.popleft()
items.pop()

print(items)
```

Output

```
deque([5, 10, 20, 30])
deque([10, 20])
```

Python list vs custom linked list vs deque

Feature	Python list	Custom linked list	deque
Underlying structure	Dynamic array	Nodes and links	Linked blocks
Node references	Not exposed	Explicit	Not exposed
Add/remove at front	Usually O(n)	O(1)	O(1)
Access by index	O(1)	O(n)	O(n)

Note: The time complexity for inserting or deleting at the end of our custom singly linked list is O(n) because this implementation does not maintain a tail reference.

12. Time Complexity

Time complexity describes how the time taken by an operation grows as the number of nodes increases.

Operation	Time complexity	Reason
Traversal / Display	O(n)	May visit every node
Search	O(n)	May check every node
Insert at beginning	O(1)	Only head and new node links change
Insert at end	O(n)	Must reach the last node
Delete at beginning	O(1)	Head moves to the next node
Delete at end	O(n)	Must reach the second-last node

13. Practice Questions

Try these questions after completing the examples. Write each operation as a method inside the LinkedList class.

• Write a method to count the total number of nodes.

• Write a method to search for a given value and print whether it is present.

• Write a method to find the largest value in the linked list.

• Write a method to find the middle node.

• Write a method to reverse a singly linked list.

• Write a method to insert a node at a given position.

• Write a method to delete a node containing a given value.

Suggested practical activity
Create a linked list containing five student marks. Display all marks, insert a new mark at the beginning, delete the last mark, and display the updated list.

14. Quick Revision

• **Node:** An object that stores data and a reference to the next node.

• **Head:** Reference to the first node.

• **Next:** Reference to the next node in the chain.

• **None:** Indicates the end of the list.

• **Traversal:** Visiting nodes one by one.

• **Insertion:** Adding a new node by adjusting references.

• **Deletion:** Removing a node by adjusting references.

Remember for exams
In a singly linked list, each node points only to the next node. The last node always points to None. Always handle the empty-list case when writing insertion and deletion methods.